COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

07

PACKAGE 8 · WORKBOOK

Lab and project workbook

20 practical labs · Five module projects · G16 ISA appendix

Undergraduate computer-engineering core

Modules 31-35 · Gate G

Edition: September 28, 2026

A guided self-study curriculum. Supplied examples are learning references, not completed learner projects. Simulation is the default; physical hardware is optional.

Contents

Page references and PDF bookmarks are linked for navigation.

Work that demonstrates understanding 3
Lab 31.1 - Loaded-divider notebook 4
Lab 31.2 - Transient and frequency-response investigation 5
Lab 31.3 - CMOS transfer and noisy-threshold study ... 6
Lab 31.4 - Noisy joystick, ADC and latency pipeline 7
Lab 32.1 - Exhaustive combinational building blocks ... 8
Lab 32.2 - Controller transition and timing workbook .. 9
Lab 32.3 - Bit-accurate arithmetic and error report 10
Lab 32.4 - Bundled-data ownership crossing model ... 11
Lab 33.1 - Counter, swap and scheduling semantics . 12
Lab 33.2 - FIFO scoreboard and reproducible failure .. 13
Lab 33.3 - Properties, covers and assumption audit ... 14
Lab 33.4 - FIFO synthesis and mapping comparison .. 15
Lab 34.1 - G16 architectural model and datapath 16
Lab 34.2 - Pipeline-valid and hazard integration 17
Lab 34.3 - Variable-latency memory and transaction ledger 18
Lab 34.4 - Twenty-program processor verification campaign 19
Lab 35.1 - Timer, sticky interrupt and MMIO side effects 20
Lab 35.2 - Raster counters, color bars and boundary-safe swap 21
Lab 35.3 - Streaming arithmetic and end-to-end cost 22
Lab 35.4 - Gate G integration review and reproducible handoff 23
Project 31: a circuit laboratory notebook 24
Project 32: specified logic and state 25
Project 33: parameterized synchronous FIFO RTL 26
Project 34: a small verified processor 27
Project 35: integration and Gate G 28
Appendix - G16 v1 instruction-set and execution contract 29

Work that demonstrates understanding

This workbook expands all 20 original practical assignments and five module projects. Preserve the original pass requirement shown with each lab. Use the procedure as a scaffold, not as permission to skip independent predictions or negative cases. The five module projects and Gate G are included in the original planning estimate.

Keep supplied examples unchanged. Work in your own directory, record provenance, and compare results against a genuinely independent calculation or model. “The script ran” is not equivalent to “the result satisfies the specification.”

For each lab retain a prediction, exact command, tool version, seed where applicable, raw result, discrepancy explanation, deliberately failing case and repaired result. The lab-record template provides the fields. Physical circuits are optional and require the safety review; hardware simulation and synthesis requirements are not replaced by a physical demo.

Lab 31.1 - Loaded-divider notebook

Original assignment and pass requirement: Simulate a two-resistor divider and verify node voltages and current conservation. Pass: predictions include units, the source/load assumptions are stated, and a safety checklist precedes any physical experiment.

Related files: `reference/python/circuits.py; reference/spice/divider.cir; templates/LAB_RECORD.md`

Predict before running

Calculate unloaded and loaded outputs for the worked circuit. Predict the sign of output change when RL is halved. Derive the two 1% resistor extremes without random sampling.

Procedure

1. Write the circuit boundary, node labels, reference directions and units in a lab record. Draw a simple connection description that another learner can reconstruct.
2. Run the Python divider example and independently reconstruct the KCL equation. Change RL to 3,000 ohms, then to 12,000 ohms; do not edit the expected answers to match the implementation.
3. Run the supplied SPICE operating-point deck when ngspice is installed. Record its actual version and sign convention for voltage-source current. The Python result is an analytical model, not SPICE evidence.
4. Add a 10-megohm measurement load to the unloaded divider and calculate the small error. Sweep the resistor tolerance corners, keeping source voltage exact.
5. Save a table of predicted voltage, observed/model voltage, current residual, power residual and discrepancy explanation. Complete the safety checklist even for a simulation-only submission.

Acceptance check

For the ideal Python model, absolute node-voltage error is at most 1e-9 V and current-balance residual at most 1e-12 A. Use a documented, looser tolerance for SPICE rather than claiming exact floating-point equality. Both tolerance extremes and the changed-load case must pass.

Deliberate failure

Replace R2 parallel RL with R2 + RL in a private copy. Show a case that detects the error and explain why the predicted direction is wrong.

Submission

31_1_divider.md, a table of at least five load cases, the simulator log when run, and one failing-test transcript followed by the repair.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 31.2 - Transient and frequency-response investigation

Original assignment and pass requirement: Simulate an RC step response and a sinusoidal frequency sweep. Pass: the measured time constant agrees with the ideal calculation within numerical tolerance, and the report distinguishes transient from steady-state behavior.

Related files: `reference/python/circuits.py; reference/spice/rc.cir; templates/LAB_RECORD.md`

Predict before running

Calculate tau, fc, t90, energy and the voltage at one tau for both zero and 1 V initial conditions. Predict how each changes when C doubles.

Procedure

1. Generate a time table from 0 to 50 ms using the analytical function. Include exact samples at tau and t90 rather than estimating them from a coarse plot.
2. Implement an Euler solver in starter work and compare dt/tau values 0.5, 0.1 and 0.01 with the exact solution. Report maximum absolute error over the same time interval.
3. Run the SPICE transient and AC deck when available. Extract the amplitude near fc and at 100 Hz. Label an AC sweep as a small-signal steady-state result.
4. Repeat with a nonzero initial condition in the mathematical model, and describe how the SPICE initialization procedure differs from simply setting the source DC voltage.
5. Write a filter tradeoff paragraph: how much noise attenuation was gained, how much step delay was added, and which omitted nonideal effects could alter a physical result.

Acceptance check

The ideal values agree within 0.1%; t90 is about 23.026 ms and fc about 15.915 Hz for the baseline. The Euler error must decrease as the step is refined over the tested stable range. Initial conditions, numerical step and units are explicit.

Deliberate failure

Use milliseconds as though they were seconds in the exponent, or omit the micro prefix on C. Show which dimensional check catches the discrepancy before plotting.

Submission

31_2_transients.md, transient and frequency tables, convergence results, and actual SPICE logs if the tool was run.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 31.3 - CMOS transfer and noisy-threshold study

Original assignment and pass requirement: Simulate a CMOS inverter and one simple amplifier or comparator stage. Pass: plot input/output behavior, identify valid operating regions and explain one failure of the idealized model.

Related files: `reference/spice/cmos_inverter.cir; reference/python/circuits.py`

Predict before running

Predict the inverter output near 0 V and 3.3 V input. Calculate the stated fictional noise margins and the non-inverting amplifier outputs before checking range limits.

Procedure

1. Run the CMOS DC-sweep deck with ngspice and export input/output data. If the simulator is unavailable, complete the analytical work now but leave the transistor-simulation requirement explicitly open.
2. Identify low-output, high-output and transition regions. Record the toy model parameters and explain why these are not a vendor-process characterization.
3. Implement the hysteretic comparator in a software model. Check the worked sequence and add noisy values near both thresholds.
4. Compare a zero-hysteresis decision with the hysteretic decision using the same input sequence. Count transitions and describe the resulting responsiveness tradeoff.
5. Build an interface worksheet containing recommended operating limits, absolute limits, guaranteed input thresholds, guaranteed output levels and stated load assumptions. Use the fictional example for calculation; do not treat it as a procurement recommendation.

Acceptance check

The simulated CMOS curve inverts over the educational range; the report identifies at least one limitation of the device model. The comparator sequence is exactly 0001110 under the defined equality convention, and an out-of-range amplifier case is rejected rather than reported as a valid output.

Deliberate failure

Replace the retained state between 1.5 and 1.8 V with an unconditional zero. Find the smallest trace that proves hysteresis has been lost.

Submission

31_3_interfaces.md, transfer data and actual simulator log, comparator trace, and the completed voltage-domain worksheet.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 31.4 - Noisy joystick, ADC and latency pipeline

Original assignment and pass requirement: Simulate a noisy joystick signal, quantization and a simple filter. Pass: report step size, calibration error, added response delay and behavior near the dead-zone boundary.

Related files: `reference/python/circuits.py; reference/python/input_demo.py`

Predict before running

Calculate LSB width, offset in codes and the alpha=0.25 90% update count. Predict which sampled cosine frequencies coincide at 1 kHz.

Procedure

1. Generate a deterministic joystick trace with a resting interval, a step, a ramp and a return to center. Add seeded noise and a known offset; preserve the clean trace separately.
2. Apply the explicitly defined ADC bins, reconstruct bin centers and document clipping. Compare error against the clean analog trace rather than against the noisy values alone.
3. Estimate center using only the designated calibration interval. Implement both hard and rescaled dead-zone behavior, including equality at the boundary.
4. Run at least two filter alpha values on identical input. Report resting variation, 10-to-90% or threshold response with an explicit time origin, and peak error during motion.
5. Generate 700 Hz and 300 Hz sampled cosines at 1 kHz. Demonstrate indistinguishability numerically, then explain what a presampling filter must accomplish.

Acceptance check

All 4096 codes are representable under the stated convention; clipping and exact boundary cases are tested. The 90% threshold is first crossed on update 9 for the ideal alpha=0.25 step. Noise reduction is reported beside added delay, and calibration is not trained on the evaluation motion.

Deliberate failure

Use 4095 in one direction and 4096 in the reverse conversion without documenting the convention. Find an endpoint or half-bin test that reveals the inconsistency.

Submission

31_4_controller.md, raw/calibrated/filtered tables, calibration interval and noise seed, and numerical aliasing comparison.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 32.1 - Exhaustive combinational building blocks

Original assignment and pass requirement: Implement a small multiplexer, decoder and four-bit adder in a logic simulator or model. Pass: exhaustive truth-table tests cover every supported input and no state is required for a result.

Related files: `reference/python/digital.py; starter/32_1_logic.md`

Predict before running

Write truth tables for the mux and full adder. Calculate all outputs for A=13, B=7 and both carry inputs before running code.

Procedure

1. Implement a mux, enabled decoder and one-bit full adder from Boolean expressions, without calling a prebuilt arithmetic operation for the adder.
2. Compose four full adders into a ripple-carry function. Keep carry-out separate from the low four result bits.
3. Write an independent integer-arithmetic oracle and exhaust all 512 adder cases. Exhaust every mux and decoder input combination.
4. Record exact case counts and assertions. Add a deliberate wrong-selected-output decoder and demonstrate why one-hotness alone fails to catch it.
5. Explain a propagation-delay or glitch case that the zero-delay model does not cover. Keep that limitation separate from the successful truth-table results.

Acceptance check

All 512 adder input cases pass, all mux and decoder cases pass, and the independent oracle does not reuse the implementation recurrence. Invalid parameters are rejected and all output widths are explicit.

Deliberate failure

Change carry-out to A AND B, ignoring carry-in. Find a smallest failing full-adder input and then the corresponding four-bit failure.

Submission

32_1_logic.md, implementation, exhaustive-test log and the injected-fault counterexample.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 32.2 - Controller transition and timing workbook

Original assignment and pass requirement: Design a controller with idle, running and done states, including reset and abort. Pass: a transition table matches simulation and a timing worksheet includes setup and hold as different checks.

Related files: `reference/python/digital.py; templates/TIMING_WORKSHEET.md`

Predict before running

Fill the full controller transition table including reset, abort, start, complete and acknowledge conflicts. Calculate the worked setup and hold slacks.

Procedure

1. Encode IDLE, RUN and DONE in a pure next-state function. Keep output calculation separate from updating state.
2. Exhaust all states and all 32 combinations of the five Boolean controls. Test an illegal state with the declared recovery rule.
3. Record pre-edge and post-edge values for normal, abort and conflicting-input traces. Ask a reviewer to predict the next state before showing the run.
4. Complete separate minimum- and maximum-delay inequalities. Recalculate at periods 8 ns and 16 ns and explain which slack changes.
5. Add an enable that holds state without creating a new clock. Define whether reset and abort override that enable, and write the corresponding tests.

Acceptance check

The full transition matrix passes under documented priorities; the worked setup slack is +0.5 ns and hold slack -0.1 ns. The explanation correctly separates minimum and maximum delay and does not claim slower clocking cures hold.

Deliberate failure

Swap abort and complete priority only in RUN. Use the conflicting-input trace to catch the change.

Submission

32_2_fsm.md, transition matrix, edge trace, timing worksheet and one priority-regression test.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 32.3 - Bit-accurate arithmetic and error report

Original assignment and pass requirement: Implement wrapping and saturating addition plus a fixed-point multiply. Pass: boundary cases are exhaustive for a small width and an error report quantifies rounding behavior against a floating-point reference.

Related files: `reference/python/fixed_point.py; reference/rtl/sat_add_stream.sv`

Predict before running

Compute eight-bit positive and negative overflow cases, the 1.5 times -0.75 fixed-point product, and both signs of the tie cases.

Procedure

1. Implement signed conversion, wrapping and saturation using explicit width checks. Exhaust every pair of four-bit signed addends.
2. Implement a widened fixed-point multiplier with nearest-even rescaling and final saturation. Do not use a floating-point operation as the bit-accurate oracle.
3. Use rational arithmetic or integer quotient/remainder reasoning for independent expected tie behavior. Test negative ties, minimum negative operands and zero.
4. Evaluate a reproducible set of 16-bit,F=8 operands against a floating-point or rational value reference. Separate clipped and nonclipped errors.
5. Write the software/hardware numerical contract, including total width, fractional bits, signedness, intermediate width, rounding order and saturation points.

Acceptance check

Every four-bit addend pair passes both policies, all directed ties match nearest-even, and clipping counts are reported separately. The exact worked product reconstructs -1.125. A report explains at least one case where reordering saturation changes the answer.

Deliberate failure

Replace nearest-even rescaling with unconditional arithmetic shifting. Identify negative and positive tie inputs that distinguish the policies.

Submission

32_3_numeric_contract.md, arithmetic tests, boundary table and a numerical-error report with seed and workload bounds.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 32.4 - Bundled-data ownership crossing model

Original assignment and pass requirement: Model a request/acknowledge crossing and explain its stability assumptions. Pass: the receiver never accepts a partially updated word in the protocol model, and the report states that simulation is not an analog metastability proof.

Related files: `starter/32_4_cdc.md; templates/CDC_RESET_REVIEW.md`

Predict before running

Explain why binary 0111-to-1000 can be incoherent if bits are synchronized independently. Write a precondition/postcondition for the request/acknowledge transfer.

Procedure

1. Implement an event-driven two-clock protocol model with separate source and destination edge schedules. Define deterministic ordering for coincident simulated edges.
2. Hold a payload stable from request launch through returned acknowledgement. Model synchronizer observation latency, not analog voltage evolution.
3. Sweep at least five phase offsets and three clock-period ratios, including source faster and destination faster. Include back-to-back requests separated by acknowledgement.
4. Inject additional allowed observation delay and a reset during an outstanding transfer. Test the explicitly chosen drop-and-reinitialize or drain policy.
5. Write safety assertions for stable payload, no duplicate acceptance and no overwrite while busy. State the clock-progress assumption needed for eventual completion.

Acceptance check

All chosen phase/ratio cases preserve the specified accepted-word sequence. Reset outcome is deterministic under the declared policy. The report explicitly says the digital protocol model does not prove analog metastability reliability or physical CDC closure.

Deliberate failure

Permit a second source write before acknowledgement. Construct a trace in which the destination receives the overwritten payload or a protocol assertion fires.

Submission

32_4_cdc.md, crossing inventory, source/destination traces, reset review and a list of modeling assumptions.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 33.1 - Counter, swap and scheduling semantics

Original assignment and pass requirement: Implement a counter and two-register swap using a simulator. Pass: tests explain old/new values at each edge, lint warnings are resolved or justified, and no unintended latches are inferred.

Related files: `reference/rtl/semantics_demo.sv; tests/tb_semantics.sv; tools/run_hdl.py`

Predict before running

Trace two cycles of the nonblocking swap from a=1,b=2. Trace a three-bit saturating counter from six with enable high, then reset.

Procedure

1. Write the predicted old and new values before opening the reference testbench. Identify the exact observation phase used by the bench.
2. Run the HDL semantics test when Icarus Verilog is installed. Capture the actual command, simulator version and output; unavailable tools mean not run, not pass.
3. Implement your own counter and swap module in starter work. Add simultaneous reset/enable and minimum-width cases.
4. Introduce a combinational next-state block and give all outputs defaults. Run lint if available, then inspect synthesis output for unintended latches.
5. Explain what the for-loop and each clocked variable imply as hardware. List simulation-only statements separately from the synthesizable module.

Acceptance check

Swap and counter edge traces match prediction, reset priority is tested and output observations are race-free. No unresolved unintended-latch or multiple-writer issue remains; every skipped tool stage is explicitly marked.

Deliberate failure

Replace the swap nonblocking assignments with blocking assignments. The bench must detect both registers becoming the same value.

Submission

33_1_rtl_semantics.md, your RTL, actual simulation/lint logs and a before/after edge table.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 33.2 - FIFO scoreboard and reproducible failure

Original assignment and pass requirement: Create a FIFO testbench with empty, full, simultaneous read/write and reset cases. Pass: an intentionally corrupted RTL version is detected, and a seed plus trace reproduces the first mismatch.

Related files: `reference/python/fifo.py; reference/rtl/sync_fifo.sv; tests/tb_fifo.sv; tools/run_hdl.py`

Predict before running

Write pre-edge push/pop and post-edge queue for empty, full and interior simultaneous cases under the conservative policy.

Procedure

1. Run the Python queue-model tests first, then the RTL FIFO bench when the simulator is available. Record which stages actually executed.
2. Create your own scoreboard using a sequence abstraction. Do not inspect DUT storage to obtain expected data.
3. Test depths 1,2,3 and 8. Force each boundary, pointer wrap, reset while nonempty and a prolonged output stall before randomized traffic.
4. Collect directed coverage counters and seed-labelled randomized traces. Preserve the first mismatch rather than only the final wrong queue.
5. Inject a documented mutation and demonstrate a nonzero test exit or a caught assertion. Restore the implementation and rerun the exact seed.

Acceptance check

Every directed event is observed at every applicable depth, sequence order is preserved and stalled output remains stable. The legal driver retains a blocked payload. At least one deliberate DUT mutation is detected with a reproducible counterexample.

Deliberate failure

Allow writes while full without changing the acceptance protocol. The full-boundary test must detect an overwritten or spuriously accepted item.

Submission

33_2_fifo_verification.md, bench and scoreboard, coverage counts, passing seeds and one mutation failure/repaired rerun.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 33.3 - Properties, covers and assumption audit

Original assignment and pass requirement: Add occupancy, ordering and stalled-data assertions. Pass: bounded results state their depth, at least two meaningful cover goals are reached, and each environment assumption is justified.

Related files: `starter/formal/fifo_formal.sv; starter/formal/fifo.sby; templates/PROPERTY_LEDGER.md`

Predict before running

State occupancy, stalled-output and ordering properties in plain language. For each, identify reset behavior and environment assumptions.

Procedure

1. Translate the occupancy and stability claims into supported assertions. Guard history and define reset semantics explicitly.
2. Add an ordering check that is stronger than occupancy. Use a reference sequence or tracked item, and explain why the method tests payload identity.
3. Add at least two reachable cover goals and document any parameter-specific impossible cover such as depth-one interior simultaneity.
4. Run bounded checking at a recorded depth, then attempt an unbounded proof where tools support it. Preserve PASS, FAIL or UNKNOWN exactly as reported.
5. Remove or weaken each assumption in turn and discuss the consequence. Inject an ordering or stability bug and inspect the earliest counterexample.

Acceptance check

The property ledger separates assumptions, assertions and covers; at least two meaningful covers are reached in an executed run. Any bounded result includes depth, and any unbounded claim includes actual engine output. Ordering is checked, not inferred from count bounds.

Deliberate failure

Add an assumption that input-valid is always zero. Show that useful cover goals fail even if some safety checks become trivially true.

Submission

33_3_properties.md, formal source, actual tool logs/status, two cover traces and an assumption-by-assumption justification.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 33.4 - FIFO synthesis and mapping comparison

Original assignment and pass requirement: Synthesize the FIFO and compare two storage implementations or depths. Pass: record device/model, tool version, inferred memory, registers and timing constraints; reconcile any post-synthesis behavior differences.

Related files: `tools/run_synthesis.py; reference/rtl/sync_fifo.sv; templates/SYNTHESIS_RECORD.md`

Predict before running

Calculate payload bits and pointer/count widths at depths three and eight. Predict why asynchronous-head-read and reset choices can affect memory mapping.

Procedure

1. Run the generic synthesis script with an installed Yosys, recording version and full commands. Keep raw reports, not only a copied total.
2. Compare W=16 at DEPTH=3 and DEPTH=8. Record abstract storage bits, inferred memories and generic cell counts under the same flow.
3. Inspect warnings, latches, multiple drivers and undriven signals. Explain each accepted warning; do not suppress the entire warning class to get a clean-looking log.
4. Design a registered-read alternative or a second storage depth and rerun functional tests under the correct interface semantics.
5. Complete the target-constraint worksheet and explicitly mark target-specific clocks, I/O, place-and-route and timing as not executed unless they actually were.

Acceptance check

Two actual synthesis reports have identical documented flow except for the intended variation; functionality is rechecked and memory mapping differences are explained. No generic cell report is called a physical LUT count, power result or routed Fmax.

Deliberate failure

Declare occupancy using ceil(log2(DEPTH)) bits at a power-of-two depth. Show how the full count becomes unrepresentable, and detect it with simulation or formal checks before trusting synthesis.

Submission

33_4_synthesis.md, two raw reports, commands/versions, parameter table and a timing/ implementation evidence-boundary statement.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 34.1 - G16 architectural model and datapath

Original assignment and pass requirement: Implement arithmetic, load/store and one branch form in a reference model and datapath. Pass: hand-encoded programs produce the expected register and memory state and unsupported encodings have defined behavior.

Related files: `reference/python/g16.py; reference/rtl/g16_core.sv; tests/test_g16.py; tools/generate_cpu_vectors.py`

Predict before running

Decode the six worked machine words manually. Predict register state, memory word 3 and PC after each retirement.

Procedure

1. Read the complete G16 v1 specification in the workbook appendix. Write a separate datapath and control table before using the supplied reference.
2. Implement an instruction-level model or independently reimplement at least the required instruction subset. Add explicit faults instead of treating unknown opcodes as NOP.
3. Implement the single-cycle or multicycle RTL datapath in learner work. State the instruction and data memory timing assumptions.
4. Run hand-encoded programs and compare each retirement event against the independent model. Check loads to r0 still perform required address validation.
5. Compare G16 word addressing, register count and unsupported features with the exact assigned RISC-V reference sections. Do not rename G16 as a RISC-V processor.

Acceptance check

Arithmetic, load/store and a branch run with matching intermediate state; reserved/illegal encodings and address faults have no unintended side effects. The ISA/microarchitecture distinction and memory assumptions are explicit.

Deliberate failure

Zero-extend the LDI immediate rather than sign-extend it. Use LDI r1,-1 to show a mismatch between 65535 and 511.

Submission

34_1_g16.md, ISA table, datapath/control description, your implementation and per-retirement traces for at least five initial programs.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 34.2 - Pipeline-valid and hazard integration

Original assignment and pass requirement: Pipeline the processor or model a pipeline with explicit valid bits. Pass: dependent arithmetic, load-use, branch-taken and back-to-back memory tests match the architectural reference.

Related files: `starter/34_2_pipeline.md; reference/python/g16.py; templates/RETIREMENT_TRACE.md`

Predict before running

Create timing tables for an arithmetic dependency, an immediate load-use dependency and a taken branch over a store. State when each result becomes available.

Procedure

1. Choose a stage partition and define every stage’s valid/data/control fields. Write hold, advance, bubble and flush rules.
2. Implement a pipeline model or RTL extension with explicit validity. Keep the instruction-level interpreter as an independent architectural oracle.
3. Add forwarding with youngest-producer priority, or conservative stalls. Test dependencies on r0, two successive writers to one register and dependent store data.
4. Test a load-use sequence with the actual modeled memory latency, then a taken branch over both a store and a register write.
5. Compare retirement sequences and final memory, and report fill/drain cycles and stall causes. Inject overlapping stall/flush conditions rather than testing them only separately.

Acceptance check

Dependent arithmetic, load-use, branch-taken and back-to-back-memory tests match the architectural reference. Invalid or flushed stages cannot write registers or memory. Cycle-count claims name the stage and memory model.

Deliberate failure

Flush instruction bits but leave the stage write-enable valid. Use a taken branch over a store to detect the surviving wrong-path side effect.

Submission

34_2_pipeline.md, stage timing tables, pipeline implementation, retirement comparison and stall/flush regression tests.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 34.3 - Variable-latency memory and transaction ledger

Original assignment and pass requirement: Add a variable-latency memory model and an optional direct-mapped cache. Pass: randomized stalls preserve program results, each accepted store occurs exactly as specified and cache tests include conflicting addresses.

Related files: `reference/python/memory_protocol.py; starter/34_3_memory.md; reference/python/g16.py`

Predict before running

Write the issued/accepted/completed/retired timeline for the worked store. Calculate the cache tag/index/offset split for the separate byte-addressed example.

Procedure

1. Implement a single-outstanding request/response wrapper with stable stalled payload and explicit WAIT_REQUEST/WAIT_RESPONSE states.
2. Integrate it with the processor or cycle model, not only a standalone memory object. Keep the architectural reference unchanged.
3. Run deterministic stall patterns followed by seeded randomized readiness and response delays. Count accepted stores independently from offered cycles.
4. Test reset before acceptance and after acceptance, choosing and documenting the exact side-effect/stale-response policy. Add invalid-address and memory-error tests.
5. For the optional cache, implement valid/tag handling and the chosen write policy, then test conflicting addresses and dirty eviction where applicable. Without a cache, complete the analytical address-split exercise.

Acceptance check

Programs preserve architectural results under randomized stalls; every accepted store has exactly the specified number of side effects. Request payload remains stable while blocked, errors do not retire successfully and reset ownership is documented. Any included cache passes conflict tests.

Deliberate failure

Leave request-valid asserted while waiting for a response and allow the memory to accept every ready edge. The store counter must detect duplicate writes.

Submission

34_3_memory.md, request/response state machine, transaction ledger, stalled-program comparisons and optional cache-policy evidence.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 34.4 - Twenty-program processor verification campaign

Original assignment and pass requirement: Run at least twenty directed programs and a reproducible randomized set against the reference. Pass: mismatch localization works, binary layout is documented and a supported compiled or assembled program runs end to end.

Related files: `tests/test_g16.py; tools/generate_cpu_vectors.py; tools/run_hdl.py; templates/RETIREMENT_TRACE.md`

Predict before running

Choose three directed programs and write expected retirement events manually: one arithmetic/store/load, one backward branch and one terminal fault.

Procedure

1. Build a test inventory of at least twenty directed programs with the intended feature or failure mode for each. Preserve hand-encoded cases independently of the assembler.
2. Run every supported case on the architectural reference and DUT. Compare per-retirement register/memory events and terminal fault/halt state.
3. Add a seeded legal-program generator with a bounded execution budget and explicit classification for halt, fault and timeout. Run at least ten recorded seeds with varied operands.
4. Inject one decode, branch-base or side-effect fault and confirm that the comparator reports the first divergent event and reproducing command.
5. Have another person run the assembled example and a failing/repair regression. Write the supported ISA, image format, tool versions and unimplemented features in the handoff.

Acceptance check

At least twenty directed programs and a reproducible randomized set match between DUT and independent reference; mismatch localization works and an assembled program runs end to end. Timeouts and unsupported cases are not counted as passes.

Deliberate failure

Compute BZ targets relative to PC rather than PC+1. A backward loop and forward skip should localize the first wrong next-PC event.

Submission

34_4_cpu_campaign.md, inventory, program images, seeds, reference/DUT traces, mismatch report and independent-run record.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 35.1 - Timer, sticky interrupt and MMIO side effects

Original assignment and pass requirement: Implement a simulated timer and memory-mapped control block. Pass: register side effects are specified, interrupts are acknowledged without losing new events and buffers remain owned until completion.

Related files: `reference/python/peripherals.py; starter/35_1_mmio.md; templates/INTERFACE_SPEC.md`

Predict before running

Trace the compare=3 timer and the clear-plus-event race. For each MMIO register, predict side effects and error cases before implementing it.

Procedure

1. Implement the timer model with the stated increment/compare order, enable behavior, wrap and synchronous reset.
2. Exhaust pending/clear/event Boolean combinations for set-wins status. Separate pending from interrupt-enable gating.
3. Create a memory-mapped control model with one accepted request and one held response. Attach a side-effect counter independent of response duration.
4. Model buffer ownership through prepared, device-owned and completed states. Reject release or CPU mutation while device-owned under the chosen contract.
5. Test invalid offsets, access directions, reserved bits, reset while outstanding and response backpressure. Preserve the accepted-transaction ledger.

Acceptance check

The timer event occurs on the defined edge, clear-plus-event preserves the new pending status, and every MMIO side effect occurs once. The ownership model rejects early reuse and the report does not confuse volatile with DMA coherence.

Deliberate failure

Make clear override a simultaneous event. The eight-case pending-bit truth table must catch the lost event.

Submission

35_1_io.md, register map, timer/IRQ traces, transaction tests and buffer-ownership state machine.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 35.2 - Raster counters, color bars and boundary-safe swap

Original assignment and pass requirement: Create a simulated scanout counter and color-bar/sprite generator. Pass: exact active dimensions and frame boundaries are verified, captured images match the reference and buffer swapping occurs only at the declared boundary.

Related files: `reference/python/video.py; reference/rtl/scanout.sv; tests/tb_scanout.sv`

Predict before running

Calculate the educational frame period, active pixel count and blanking count. Compute the separate 320-by-240 RGB565 storage and visible-read bandwidth.

Procedure

1. Run the software scanout model and capture the reference PPM. Check exact coordinate order and active count, not only whether the image opens.
2. Implement or extend RTL counters and color generation. Run at least three consecutive frames to expose wrap errors.
3. Compare every active pixel with the independent reference while separately checking all timing periods and boundary pulses.
4. Add a one-stage pixel pipeline and delay its sideband signals consistently. Introduce a partially off-screen sprite or a pending frame-boundary buffer swap.
5. Test requests immediately before, at and after the selected frame boundary. Record the exact accepted/deferred policy and detect any within-frame source change.

Acceptance check

Each educational frame has 4,800 periods and 3,072 active pixels with exact wrap. Captured active pixels match the reference and any buffer swap occurs only at the declared boundary. Physical video compatibility is not claimed from these timings.

Deliberate failure

Use <= rather than < for one active-region bound, or delay color without delaying active. The count or pixel-alignment test must fail.

Submission

35_2_scanout.md, timing and pixel comparisons, simulated/reference captures, and boundary/sprite tests.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 35.3 - Streaming arithmetic and end-to-end cost

Original assignment and pass requirement: Build a streaming arithmetic or pixel block plus a software driver model. Pass: outputs match the reference over boundary cases, backpressure preserves order and end-to-end cost includes setup and transfer.

Related files: `reference/rtl/sat_add_stream.sv; tests/tb_sat_add.sv; reference/python/fixed_point.py`

Predict before running

Calculate positive and negative saturation cases, then solve the illustrative batch break-even inequality before evaluating a table.

Procedure

1. Write the streaming numerical and handshake contract, including replacement when a blocked/ output slot becomes available and reset flushing.
2. Implement the arithmetic block and an independent scoreboard keyed to accepted transactions. Test boundaries before random operands.
3. Exercise long output stalls, input gaps, simultaneous output consumption/input replacement and reset while work is pending.
4. Measure modeled cycle throughput separately from actual simulator runtime. Do not call simulation wall time accelerator execution time.
5. Build an end-to-end cost table for small, break-even and large batches. Include setup and transfers, label illustrative coefficients and compare equivalent numerical quality.

Acceptance check

Outputs match the bit-accurate reference; order, no-duplication and stalled-data stability hold. The worked cost model ties at N=5,600, and modeled versus physical timing is explicit. Any speedup claim includes overhead and application fraction.

Deliberate failure

Perform the sum in 16 bits before sign-extending it to 17. The positive and negative endpoint tests must catch incorrect saturation.

Submission

35_3_accelerator.md, numeric/interface contract, transaction scoreboard logs and a cost/break-even analysis.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Lab 35.4 - Gate G integration review and reproducible handoff

Original assignment and pass requirement: Complete Gate G with an integration test, synthesis report and short demonstration. Pass: another person reruns the simulation, all timing assumptions are listed and any physical measurements are distinguished from modeled estimates.

Related files: `docs/GATE_G_CAPSTONE.pdf; source/GATE_G_CAPSTONE.md; templates/GATE_G_REVIEW.md; starter/gate_g/`

Predict before running

Select the subsystem and list five claims that would have to be true for software to trust it. Map each to a proposed test or analysis before integration.

Procedure

1. Freeze the first interface revision and numerical/reset assumptions. Reuse module artifacts but identify every learner-authored change.
2. Integrate the RTL and independent software reference, then run boundary, randomized, backpressure, invalid-access and pending-reset tests.
3. Run synthesis and preserve actual reports. Complete a claims table that distinguishes software analysis, RTL simulation, formal results, generic synthesis and physical measurement.
4. Ask another person to reproduce the simulation from the README and review one waveform plus one failing/repair regression.
5. Complete the oral defense and a live parameter or interface change. Close critical findings, and list remaining bounded limitations without relabeling them as successes.

Acceptance check

Gate G evidence and rubric pass with no critical correctness, reproducibility or claim-integrity failure. The module-level CPU requirement is separately complete. Board operation is optional; actual core simulation and synthesis are not.

Deliberate failure

Use a stale passing synthesis report from before an interface change. Demonstrate how source hashes and reproducible commands expose the mismatch.

Submission

Gate G source archive, specification, independent reference, test and synthesis logs, claim-evidence map, review record and short demonstration script.

Record: first attempt date; study complete; observed evidence tier; lab pass/revision; reviewer; unresolved limitations.

Project 31: a circuit laboratory notebook

Produce one coherent notebook linking a loaded divider, an RC network, a transistor-interface study and a sampled controller input. The original project requires prediction, simulation and a safety boundary; a physical build is optional and requires the stated review.

Milestone A - conservation and loading. Derive the loaded divider from KCL, sweep at least five loads and calculate resistor/source power. Evaluate all relevant tolerance corners, then explain why a random tolerance sweep is not the same proof.

Milestone B - time and frequency. Derive the RC differential equation, calculate tau, t90 and fc, and compare an exact solution with at least three Euler time steps. Include a nonzero initial voltage. Where SPICE is available, add its transient and AC result without relabeling Python output as SPICE.

Milestone C - interface limits. Explain a toy CMOS inverter's transition, document threshold assumptions and show a comparator with hysteresis. Use a fictional datasheet table to calculate noise margins, then identify which real-device constraints would still need verification. The level-1 transistor deck is not a fabrication-process model.

Milestone D - sampled input. Compare quantized and filtered samples, demonstrate the 300/700 Hz cosine ambiguity at 1 kHz, and measure the alpha=0.25 unit-step delay. State an analog antialiasing requirement rather than claiming a digital filter removes prior aliasing.

Review evidence. Submit equations with units, independent predictions, reproducible commands, raw outputs, a numerical-error table and one deliberate model mutation. Pass only when each lesson's criteria are met and you can explain a disagreement without changing the original prediction after the fact. The reviewer asks why the divider sags under load and why a stable numerical method still has error.

Project 32: specified logic and state

Build an arithmetic/control model from truth tables and transition rules. Include a 4-bit adder, a decoder, the IDLE/RUN/DONE controller, signed fixed-point arithmetic and a synchronous queue specification. This project establishes the behavior later described in RTL.

Milestone A - exhaustive combinational reasoning. Verify all 512 adder input/carry combinations. Check both decoder one-hotness and code-to-output correspondence. Include one corrupted implementation that passes a weak property but fails a complete truth-table check.

Milestone B - state and timing. Write the FSM transition table before coding. Cover all control combinations, reset and abort priority, illegal-state recovery and DONE/start behavior. Complete both a setup worksheet and a hold worksheet with explicit sign/skew assumptions.

Milestone C - number format. Specify W and F, including whether W includes the sign bit. Test positive and negative overflow, wrap versus saturation and nearest-even ties. Check results against integer/rational arithmetic rather than an implicitly rounded floating-point reference.

Milestone D - protocol boundaries. Specify the queue's conservative policy and complete depth-one, full, empty and simultaneous-operation traces. Design a bundled-data request/acknowledge crossing on paper and in a protocol model. Vary relative clock phase and latency; state why that model does not prove analog metastability rates or routed data settling.

Review evidence. Submit truth/transition tables, exhaustive logs, fixed-point boundary tests, timing worksheets, queue traces and a CDC/reset review. Explain one counterexample for independently synchronized bus bits. Passing this project requires both logical correctness and a clear statement of the abstraction boundary.

Project 33: parameterized synchronous FIFO RTL

Implement a width/depth-parameterized FIFO from the Module 32 contract. Use a Python sequence oracle or an independently structured C++ model. The supplied FIFO is a reading/reference example; your submitted work must demonstrate independent understanding and a reviewed implementation or substantial documented modification.

Milestone A - scheduling. Predict old-versus-new values at a rising edge. Write combinational defaults and clocked state updates intentionally. Explain why RAM payload need not be reset when validity is cleared, and why control must not depend on unknown startup state.

Milestone B - verification. Run directed and seeded random cases at depths 1, 3 and 8. Check all accepted output words, count bounds, blocked-output stability and reset while work is pending. Keep the producer protocol valid; changing data while blocked is not a legitimate DUT test unless explicitly labeled a protocol-violation experiment.

Milestone C - properties and covers. Extend the formal starter beyond occupancy and stability with ordering or a tracked-item property. List environmental assumptions separately from DUT assertions. Execute reachability covers, preserve failed traces and distinguish bounded results from unbounded proof. Tool limitations belong in the report, not in a fabricated proof claim.

Milestone D - synthesis. Run generic synthesis and inspect warnings/netlist structure. Compare at least two depths and report exact tools/options. Explain why abstract payload bits, inferred generic cells and device block-RAM allocation are different quantities. A target frequency is not measured by counting cells.

Review evidence. Submit the interface contract, RTL, independent model, scripts, test log, waveform, property ledger and actual synthesis reports. Deliberately break pointer wrap or ready logic and show that a test fails before the repair. A visually plausible waveform is insufficient without an automated order check.

Project 34: a small verified processor

Implement the documented G16 v1 subset or another reviewed, equally explicit subset with a compatible independent reference. The included reference examples are G16-specific. Do not silently mix its word-addressed PC or encodings with RISC-V conventions.

Milestone A - instruction contract and baseline. Freeze architectural state, encodings, reserved fields, address units, illegal behavior, reset and retirement. Hand-encode the six-instruction example. Run the same assembled code on your model and RTL and compare committed effects.

Milestone B - pipeline reasoning. Pipeline the processor or model the pipeline with explicit valid bits as the original lesson allows. Document stage boundaries, forwarding and stalls. Demonstrate register dependencies, load-use, taken and untaken branches, a wrong-path store and structural conflicts. A model-only pipeline extension must be labeled as such; the project still requires a processor RTL implementation.

Milestone C - delayed memory. Add a variable-latency model and integrate the request/response contract. Preserve requests until accepted, commit each accepted store once and retain response ownership until completion. Include randomized request readiness and response delay. A direct-mapped cache is an extension, not a substitute for protocol correctness.

Milestone D - architectural regression. Run at least 20 directed programs plus a reproducible randomized set against an independent reference. The package provides 32 directed and 20 generated random comparison cases as a starting bank. Add cases tied to your own pipeline and memory hazards. Compare retirement streams and terminal memory/register/fault state; a common final register value alone can hide wrong-path stores.

Review evidence. Submit ISA revision, assembler/image format, baseline and extension boundaries, RTL/model code, exact test commands, first-mismatch localization and a supported assembled program running end to end. Keep unsupported instructions explicit. Explain a faulting load to r0 and why it still requires address validation. This module remains required regardless of the Gate G subsystem choice.

Project 35: integration and Gate G

Build one bounded game-oriented subsystem with a software contract. Read the separate Gate G specification and choose either its command-queue project or a reviewed equivalent. Reuse earlier module artifacts rather than treating every lesson as an unrelated demo.

Milestone A - device semantics. Specify aligned offsets, read/write side effects, exactly-once acceptance, sticky events, response hold and reset. Implement a timer/MMIO model and demonstrate a new event concurrent with clear in the declared event logic. Model CPU/device buffer ownership; a variable named volatile is not a complete DMA synchronization protocol.

Milestone B - timed pixels. Verify exact raster totals and active dimensions in simulation. Compare a captured active image with the software reference. Keep RGB, active, coordinates and buffer-swap policy aligned. The supplied 80x60 educational raster is not a documented physical display mode.

Milestone C - bounded acceleration. Implement and test a streaming arithmetic or pixel block with backpressure and explicit numeric policy. Include setup, transfer and completion costs in the model. Evaluate at least one small case where the accelerator loses and one larger case where it can win under stated assumptions.

Milestone D - design review. Complete the Gate G acceptance matrix, actual RTL regression and synthesis report. Explain which evidence is analytical, simulated, formally checked, structurally synthesized or physically measured. A board remains optional; any board work follows the safety plan.

Review evidence. Submit the chosen capstone, integration driver, reference comparison, negative tests, resource interpretation and a reproducible demonstration. Use the Gate G 100-point rubric and critical blockers. A favorable benchmark or a lit display cannot compensate for duplicated transactions, invalid reset behavior or missing processor work.

Appendix - G16 v1 instruction-set and execution contract

Scope and identity

G16 v1 is an original teaching ISA authored for this package. It is **not RISC-V**, not a commercial processor and not a full-ISA compliance claim. The RISC-V material remains assigned comparative reading about the distinction between architecture and implementation. Do not use a RISC-V compiler's machine code as G16 input. [R35]

The reference Python interpreter and single-cycle RTL implement this bounded contract. The learner must still build or modify an implementation, independently verify it, and complete the pipeline and variable-latency-memory work required by the lessons. The reference's combinational memory ports do not model synchronous FPGA block RAM.

Architectural state

Eight registers hold unsigned 16-bit bit patterns. Arithmetic interprets them modulo 65536 unless an instruction explicitly sign-extends an immediate. Register r0 always reads zero and discards register writes; it does not suppress address checks or instruction faults.

The PC is a 16-bit **word address**. Code contains 256 separate 16-bit words at addresses 0 through 255. Data contains another 256 words at addresses 0 through 255. Instruction and data spaces are separate, so a data store cannot modify code in this baseline. An image shorter than 256 words is padded with NOP words by the vector writer. The execution budget prevents an unintended NOP stream from running forever.

Reset sets PC, registers, stopped state and retirement count to zero. The core itself does not own the external data RAM; the reference testbench initializes RAM to zero for each test. A system-level reset policy for external memory must be specified separately. Reset suppresses retirement and writes.

Instruction encodings

Bits are numbered 15 down to 0. Fields are integers, not textual register names.

Instruction	Opcode and fields	Semantics
NOP	Exactly 0x0000	No side effects; PC := PC+1
LDI rd, imm9	op=1; rd[11:9]; imm[8:0]	Sign-extend imm9 to 16 bits; write rd
ADD rd, ra, rb	op=2; rd[11:9]; ra[8:6]; rb[5:3]; low[2:0]=0	rd := (ra+rb) mod 65536
SUB rd, ra, rb	op=3; same register fields and reserved bits	rd := (ra-rb) mod 65536
LD rd, base, off6	op=4; rd[11:9]; base[8:6]; off[5:0]	Load data at validated effective address
ST src, base, off6	op=5; src[11:9]; base[8:6]; off[5:0]	Store src at validated effective address
BZ reg, disp9	op=6; test register[11:9]; disp[8:0]	If register is zero, PC := PC+1+sext(disp9)

Instruction	Opcode and fields	Semantics
JMP address8	op=7; reserved[11:8]=0; target[7:0]	PC := unsigned absolute word target
HALT	Exactly 0xF000	Retire once; PC := PC+1; stop

Immediate ranges are -256 through 255 for signed nine-bit values and -32 through 31 for signed six-bit values. Branch targets and sequential PC updates wrap to 16 bits; an out-of-range fetch faults on the next attempted fetch. Effective address is `(base + sign_extend(off6)) & 0xFFFF`; the result must then be in 0..255. A wrapped result of zero is legal. There are no byte enables, unaligned accesses, privilege levels, interrupts or address translation in this baseline.

Faults are not retirements

The following terminate execution without retiring the faulting instruction: an instruction fetch outside code addresses 0..255; a data address outside 0..255; or an illegal encoding. Illegal encodings include opcodes 8 through E, nonzero low bits in NOP, nonzero reserved low three bits in ADD/SUB, nonzero reserved JMP bits, and any opcode-F word other than F000.

On a fault, the PC remains at the faulting instruction or invalid fetch address, and no register or data-memory side effect from that instruction occurs. Previously retired effects remain. In particular, `LD r0` with an invalid address faults even though the destination is the constant-zero register. Error priority in the reference is fetch address first, then instruction decode, then data address for valid memory opcodes.

The RTL terminal fault codes are 1=fetch address, 2=illegal instruction, 3=data address. Code zero means no fault. A terminal status must be compared separately from the retirement stream so that a prematurely stopped design cannot pass by emitting an empty trace.

Retirement trace schema

Each accepted architectural instruction emits exactly one row. Fields are decimal integers in CSV order:

```
index,pc,instr,next_pc,reg_we,rd,reg_value,
mem_we,mem_addr,mem_value,halt
```

`index` begins at zero. Register-write fields are enabled only for a real write to r1..r7. Disabled register fields use zero for rd and value. Store fields are enabled only for a committed store; disabled memory address and value are zero. Loads are represented through the register-write result, not a memory-write event. HALT emits `halt=1` once. NOP still retires.

Comparing equal retirement indices is necessary but not sufficient: compare every enabled effect, terminal fault, final PC, all registers and final data RAM. A stalled or pipelined implementation may take a different number of cycles. It may not reorder committed effects under this simple in-order contract.

Worked program and independent encoding

```
LDI r1, 5       ; 1205
LDI r2, 7       ; 1407
ADD r3, r1, r2  ; 2650
ST  r3, r0, 3   ; 5603
LD  r4, r0, 3   ; 4803
HALT            ; F000
```

After six retirements, r3=12, r4=12 and data[3]=12. PC=6, stopped=true and fault is absent. For ADD, 0x2000+(3<<9)+(1<<6)+(2<<3)=0x2650. This hand calculation is independent of the assembler's field-composition code.

Assembly syntax and limitations

The included two-pass assembler accepts case-insensitive operation/register names, comma-separated or whitespace-separated operands, semicolon or hash comments and case-sensitive labels. A label refers to a word address. `BZ r1, label` computes displacement from PC+1; a numeric BZ operand is already a signed relative displacement. `JMP label` uses an absolute word address. A `.word` directive admits a 16-bit literal for deliberate illegal-instruction tests.

Duplicate/invalid labels, invalid registers, out-of-range immediates, unknown mnemonics, wrong operand counts and images exceeding 256 words are errors. There is no linker, C calling convention, stack convention, binary executable format or operating-system ABI. A supported assembled program satisfies the course's compiled-or-assembled requirement; claiming a general compiler toolchain would require additional work.

Baseline and extension evidence

`reference/python/cpu_cases.py` defines 32 directed case specifications. `tests/test_g16.py` checks those plus independent hand encoding, fault non-retirement, signed extension, execution limits and 100 seeded random arithmetic programs. The HDL comparison vector generator writes 32 directed plus 20 random cases with seed 7344. Those expected files are software output until an actual RTL comparison runs.

The provided HDL runner compiles the simple core with Icarus, compares retirement CSVs, then checks terminal PC, faults, registers and all data memory. That runner was not executed in the authoring environment. Its presence is not evidence of a passing RTL core.

For the pipeline extension, choose and document stage boundaries, valid bits, forwarding paths, load-use stall rules and branch-resolution timing. For variable-latency memory, define request and response acceptance and preserve a request's payload until acceptance. Integrate the protocol without replaying accepted stores. Report baseline and extended implementations separately.